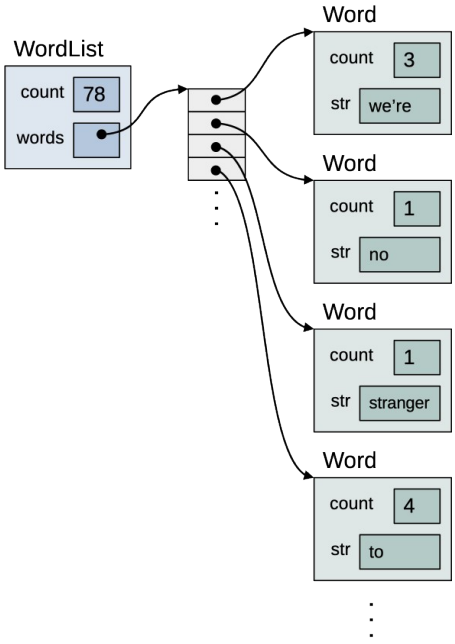
Exercise 16

Debugging with Valgrind

On the course homepage, you'll find a source file, wordlist.c, a text input file, input.txt, and an expected output file. You can download them from there, or, if you're logged in on remote.eos.ncsu.edu, you can also copy them with the following shell command:

```
cp /mnt/coe/workspace/csc/CSC230/ex/exercise16/* .
```

This program expects a filename as a command-line argument. It reads all the words from the file, storing each word in a dynamically allocated struct, **Word**, along with a count of the number of occurrences of that word in the input file. The program also uses a dynamically allocated instance of a **WordList** struct containing an array of pointers to **Word** instances, one instance of **Word** for each different word in the input file. The following figure shows how this data structure is supposed to work for the sample input file provided with the exercise. This input contains more than 75 different words, so the figure shows just the first few.



A word is a sequence of up to 8 characters, consisting of letters or the apostrophe character. Words are converted to lower-case as they're stored, and words longer than 8 characters are truncated to just the first 8. After reading in all the words, the program sorts the list of words lexicographically using an insertion sort. Finally, it prints out all the words from the input file, reporting the number of occurrences of each word.

When it's working, you should be able to run the program on the sample input file as follows:

```
$ ./wordlist input.txt
a (7)
aching (2)
and (16)
any (1)
around (6)
ask (1)
been (4)
… several lines omitted ...
what's (2)
```

```
wouldn't (1)
you (41)
you're (3)
your (2)
```

This program is already implemented for you, but it contains some bugs. You're going to use valgrind to help you find and correct the bugs.

If you run this program normally, you may noice that the program produces the right output, even though it has several bugs. This is typical of some types of bugs. If, for example, you forget to initialize a variable, it's still possible for the variable to start out with the value you want it to (maybe zero). If you index an array out of bounds, maybe you're lucky and this doesn't actually overwrite other important data you have stored in memory. Of course, correct execution of a program shouldn't depend on being lucky.

Debugging with Valgrind

This program has some bugs that you need to fix. You're going to fix them with help from valgrind. Compile the program the way you normally would. Be sure to use the -g compiler option, so valgrind can give you more detailed information about where it catches your program doing something wrong.

Then, try debugging the program with help from valgrind. You should be able to find and fix the following bugs. Be sure to re-compile the program after fixing each bug.

- Run the program in valgrind, using the default tool (memcheck) and giving it the input.txt file as a command-line argument:

  ```
  valgrind ./wordlist input.txt
  ```

  This will probably generate a lot of output, from the program and from valgrind. Scroll back and have a look at the first error reported by valgrind. It should be a complaint about a conditional jump or move depending on an uninitialized value. This means that you're using a variable, a field or an array element that never got initialized, and you have no guarantee about what value it contains. If you compiled this program with the -g flag, the next output line from valgrind should tell you where you're accessing this uninitialized value.

  It's a field in the **WordList** struct. Look back at the code in readWords() where this struct first gets allocated. Does it have any fields that didn't get initialized? Add code to initialize this field appropriately. To do this, you may need to read some of the code and make sure you understand what **readWords()** and **findWord()** are doing.

- Re-compile the program after you fix the first bug and then run it again in valgrind. Scroll back and look at the first error reported by valgrind. It should be an invalid write. The program is writing past the end of a dynamically allocated block of memory. This means either the block of memory being allocated isn't big enough for what the program is trying to store, or there's a bug in the program that's causing it to try to write more to that block than it should.

  In this case, the problem is that the dynamically allocated block of memory isn't big enough. The report from valgrind tells you two things. First, it gives a stack trace that tells you where you tried to write past the end of the block. Then, it tells you were in the code you allocated that block. Go to the line of code where the program allocated the block of memory and look carefully. This should be code to allocate an instance of the **Word** struct on the heap, but it contains a small error. Figure out what the error is and then correct it.

- After fixing this bug and re-compiling, try the program again with valgrind. Its first complaint be an invalid write, but it should be for a different place in the program. Look at the two parts of the program reported by valgrind, the place where a block of memory is used and the place where that block of memory is allocated. Figure out why the program is writing past the end of the block and fix the error.

- Rebuild the program and run it again in valgrind. The first error reported by valgrind should be another complaint about an invalid write, but it should be in another part of the program, the part where you're using a resizable array. The starter code support for growing this resizable array as needed, but there must be some kind of error. Have a look at the code and see if you can fix the error. If it's not clear what's going wrong, you can run the program in gdb and put a breakpoint right after you read a word. Stepping through the execution one statement at a time give you a chance to see where the resizable array code is going wrong.

- After you've fixed the previous error, try rebuilding the program and running it in valgrind. If your corrections are all good, you shouldn't have any more memory errors reported, but there are still a couple of things to fix. You'll just need to ask valgrind for some more detailed output.

  If you add the valgrind --track-fds=yes option, it will report on any open files when the program exits. Make sure you give this as an option to valgrind, not as an option to the wordlist program. Check back with the lecture material if you're not sure how pass command-line options to valgrind.

  Remember that your program automatically starts up with three files open, standard input, standard output and standard error. You're not expected to close these, so it's normal for a program to still have three open files when it terminates. Any additional files should be closed before the program exits.

  With the --track-fds=yes option, valgrind will give you a stack trace to tell you where you open any files that are subsequently leaked. Look at the valgrind output and see where the leaked file is being opened. Consider how the program might be getting out of the function it's in without closing this file and add code to make sure it gets closed.

- Rebuild the program and run it again with valgrind. It shouldn't report any errors or file leaks, but, near the end of the output, you should have a report of a leaked block of memory, By default, valgrind doesn't tell you much about what's wrong. Try running again with the --leak-check=full option given to valgrind. As with the --track-fds=yes option, make sure you give this as an option for valgrind, not an option for the wordlist program.

  With the --leak-check=full option, valgrind should tell you where the program allocated the block of memory that never got freed. Add code to free this block before exiting.

After fixing these bugs, your program should produce the correct output, and it should run in valgrind without any error messages, even when you ask it to report on leaked memory and open files.

Submitting Your Work

Once you've fixed all the bugs, submit your modified source file to the exercise_15 assignment in Moodle. We'll test it by making sure it produces the right output and valgrind no longer has anything to complain about.

Don't forget to make sure your output is right. In the past, some students have lost most (or all) of their points for this exercise because they concentrated on fixing the memory errors without checking to make sure the program's output is right. This exercise includes an expected output file, so you can make sure your output is correct. The grading script first checks to make sure you have the right output. If your output isn't right, you won't be able to earn any credit for fixing bugs in the program.